

ECOM EXPERTS

Odoo mirakl

Manual de usuario, técnico y de desarrollo

odoo_mirakl · 08/09/2026

Contenido

1. CONTEXTO Y OBJETIVO

2. ARQUITECTURA DE DATOS (MODELOS)

2.1 mirakl.backend

2.2 mirakl.delivery.type.mapping (Mapeo de Entrada)

2.3 mirakl.delivery.carrier.mapping (Mapeo para Tracking)

2.4 sale.order (Herencia)

2.5 stock.picking (Herencia)

3. LÓGICA DE NEGOCIO Y API (FASETAS DE DESARROLLO)

FASE 1: Servicio API (Python Class)

FASE 2: Importación de Pedidos

FASE 3: Sincronización de Salida (Eventos)

4. INTERFAZ DE USUARIO (UX)

5. REQUISITOS DE DEPURACIÓN (DEBUGGING)

1. CONTEXTO Y OBJETIVO

Este módulo replica la funcionalidad de un conector de PrestaShop existente. El objetivo es sincronizar múltiples Marketplaces basados en Mirakl con Odoo, gestionando: - Configuración de múltiples backends. - Mapeo dinámico de transportistas de entrada (Shipping Types) y salida (Carriers). - Importación automatizada de pedidos, registros de clientes y direcciones. - Sincronización de Tracking y estado de envío hacia Mirakl. - Funcionamiento unicamente para Odoo 17,18 y 19.

2. ARQUITECTURA DE DATOS (MODELOS)

2.1 mirakl.backend

- `name` : Nombre del Marketplace.
- `api_url` : URL Base del endpoint (ej. `https://marketplace.mirakl.net/api/`).
- `api_key` : Token para Authorization Header.
- `import_on_state` : Estados (SHIPPING, WAITING_ACCEPTANCE).
- `auto_accept` : Boolean.
- `last_sync_date` : Datetime de última importación exitosa.

2.2 mirakl.delivery.type.mapping (Mapeo de Entrada)

- `backend_id` : Relación con el backend.
- `mirakl_type_code` : Código del tipo de envío en Mirakl (ej. 'standard').
- `mirakl_type_label` : Nombre del tipo (ej. 'Envío Estándar').
- `mirakl_zone_code` : Código de zona de Mirakl.
- `odoo_carrier_id` : Relación con `delivery.carrier`.

2.3 mirakl.delivery.carrier.mapping (Mapeo para Tracking)

- `backend_id` : Relación con el backend.
- `mirakl_carrier_code` : Código técnico del operador Mirakl (ej. 'glsespana').
- `mirakl_carrier_label` : Nombre del operador.
- `odoo_carrier_id` : Relación con `delivery.carrier` en Odoo.

2.4 sale.order (Herencia)

- `mirakl_order_id` : Char (ID de pedido original).
- `mirakl_backend_id` : Relación al backend.

2.5 stock.picking (Herencia)

- `mirakl_tracking_sent` : Boolean.

3. LÓGICA DE NEGOCIO Y API (FASETAS DE DESARROLLO)

FASE 1: Servicio API (Python Class)

Implementar una clase de servicio que maneje: - `GET /api/orders` : Con filtros de fecha y estado. - `GET /api/shipping/types` : Para poblar el mapeo de entrada. - `GET /api/shipping/carriers` : Para poblar el mapeo de seguimiento. - `PUT /api/orders/{id}/accept` : Aceptación de líneas de pedido. - `PUT /api/orders/{id}/tracking` : Envío de número de seguimiento. - `PUT /api/orders/{id}/ship` : Marcar como enviado.

FASE 2: Importación de Pedidos

1. **Detección de Pedido Existente:** Verificar `mirakl_order_id` para evitar duplicados.
2. **Registro de Partner:** - Buscar por Email (`{order_id}@mirakl.com`). - Mapear direcciones respetando el país e intentando coincidir la provincia/estado.
3. **Mapeo de Productos:** - Buscar en `product.product` por `Internal Reference` (`offer_sku`) o `Barcode` (`product_sku`).
4. **Precios e Impuestos:** - Forzar el precio enviado por Mirakl en `sale.order.line` . - Aplicar impuestos basados en si el precio es Tax-Incl o Tax-Excl según config del backend.
5. **Transporte de Entrada:** Asignar el `delivery_carrier` basado en el mapeo configurado para el `shipping_type_code` .

FASE 3: Sincronización de Salida (Eventos)

- Hook en la validación del Albarán (`stock.picking`).
- Al validar (`done`):
 - Extraer `carrier_tracking_ref` .
 - Buscar mapeo en `mirakl.delivery.carrier.mapping` .
 - Si se encuentra, enviar a Mirakl el código del transportista y el tracking.

4. INTERFAZ DE USUARIO (UX)

- Botones en los modelos de mapeo: "Actualizar desde API". Estos botones deben llamar al servicio API y sincronizar las opciones de transportistas actuales del Marketplace.
- Pestaña Mirakl en Pedidos de Venta para visualizar el ID original.
- Vista de configuración centralizada por Marketplace.

5. REQUISITOS DE DEPURACIÓN (DEBUGGING)

- Implementar un sistema de logging que guarde las peticiones JSON completas y sus respuestas en ficheros físicos dentro de una carpeta `/logs` del módulo, activable mediante un campo `debug_mode` en el backend.